TP 1 : Programmation Orientée Objet

Classes, attributs, méthodes et héritage

Code source

On considère le programme Java suivant :

```java
import java.util.*;
class Animal {
    String name;
    Animal(String name) {
        this.name = name;
        System.out.println("new Animal " + this.name);
    }
    void sound() {
        System.out.println("Animal Sound " + this.name);
    }
}

class Dog extends Animal {
    Dog(String name) {
        super(name);
        System.out.println("new Dog " + this.name);
    }
    @Override
    void sound() {
        System.out.println("Dog Sound " + this.name);
    }
}
public class Main {
    public static void main(String[] args) {
        Animal a1 = new Animal("a1");
        Animal a2 = new Animal("a2");
        Dog d1 = new Dog("d1");
        Dog d2 = new Dog("d2");
        a1.sound();
        a2 = a1;
        a1.name = "new a1 object name";
        a1.sound();
        a2.sound();
        d2 = new Dog(d1.name);
        d1.name = "new d1 object name";
        d1.sound();
        d2.sound();
    }
}
```

Exercice 1 : Analyse des classes

▶ 1.1 Extraction des classes

Identifiez toutes les classes déclarées dans le programme.

N°	Classe
1	
2	
3	

▶ 1.2 Attributs

Pour chaque classe, indiquez les attributs déclarés dans la classe.

Classe	Attribut(s)
`Animal`	
`Dog`	
`Main`	

▶ 1.3 Méthodes

Complétez le tableau suivant avec les méthodes déclarées dans chaque classe.

Classe	Méthode(s)
`Animal`	
`Dog`	
`Main`	

▶ 1.4 Constructeurs

Identifiez le constructeur de chaque classe qui en possède un.

Classe	Constructeur
`Animal`	
`Dog`	
`Main`	

▶ 1.5 Héritage

Répondez aux questions suivantes.

a) Quelle classe hérite d'une autre classe ?

.....

b) Quelle est la classe mère ?

.....

c) Quelle est la classe fille ?

.....

d) Quelle instruction indique l'héritage dans le code ?

.....

▶ 1.6 Redéfinition de méthode

La classe `Dog` contient l'annotation :

```
1 @Override
```

a) Quelle méthode est redéfinie ?

.....

b) Dans quelle classe cette méthode est-elle définie initialement ?

.....

c) Pourquoi utilise-t-on `@Override` ?

.....

.....

Exercice 2 : Lecture du programme

À partir des instructions suivantes :

```
Animal a1 = new Animal("a1");
Animal a2 = new Animal("a2");

Dog d1 = new Dog("d1");
Dog d2 = new Dog("d2");
```

▶ 2.1 Objets

Combien d'objets sont créés par ces quatre instructions ?

Réponse : .....

▶ 2.2 Références

Donnez les références utilisées pour accéder aux objets créés.

.....

.....

▶ 2.3 Constructeurs

Lors de l'exécution de :

```
Dog d1 = new Dog("d1");
```

Quel constructeur est exécuté en premier ?

.....

Expliquez brièvement pourquoi.

.....

.....

Exercice 3 : Références et objets

Considérons l'instruction suivante :

```
a2 = a1;
```

▶ 3.1

Cette instruction crée-t-elle un nouvel objet ?

☐ Oui ☐ Non

► 3.2

Après cette instruction, `a1` et `a2` font-ils référence au même objet ?

☐ Oui ☐ Non

► 3.3

Représentez graphiquement la situation de `a1` et `a2`.

```
a1  ::::->


a2  ::::->
```

Exercice 4 : Analyse de `sound()`

Considérez les instructions suivantes :

```
1 a1.sound();
2 d1.sound();
```

▶ 4.1

Quelle version de `sound()` est appelée pour `a1` ?

.....

▶ 4.2

Quelle version de `sound()` est appelée pour `d1` ?

.....

▶ 4.3

Quel concept de la programmation orientée objet permet ce comportement ?

.....

Exercice 5 : Trace d'exécution

Donnez, ligne par ligne, l'affichage produit par l'exécution complète de la méthode `main`.

Exercice 6 : Encapsulation, getters et setters

On modifie maintenant les classes `Animal` et `Dog` afin de protéger l'attribut `name` :

```
class Animal {
    private String name;

    Animal(String name) {
        this.name = name;
        System.out.println("new Animal " + this.name);
    }

    public String getName() {
        return this.name;
    }

    public void setName(String name) {
        if (name != null && !name.isEmpty()) {
            this.name = name;
        }
    }

    void sound() {
        System.out.println("Animal Sound " + this.name);
    }
}

class Dog extends Animal {
    Dog(String name) {
        super(name);
        System.out.println("new Dog " + getName());
    }

    @Override
    void sound() {
        System.out.println("Dog Sound " + getName());
    }
}
```

▶ 6.1 Modificateur d'accès

Quel mot-clé protège l'attribut `name` et que signifie-t-il ?

.....

.....

▶ 6.2 Identification

Identifiez le getter et le setter de la classe `Animal`.

Rôle	Méthode
Getter	
Setter	

▶ 6.3 Rôle

Quel est le rôle d'un getter ? Quel est le rôle d'un setter ?

.....

.....

▶ 6.4 Accès depuis `Main`

Dans `Main`, l'instruction suivante est-elle encore autorisée avec cette nouvelle version d'`Animal` ?

```
1 a1.name = "new a1 object name";
```

Justifiez, puis écrivez l'instruction à utiliser à la place.

.....

.....

▶ 6.5 Classe `Dog`

La classe `Dog` utilise `getName()` au lieu de `this.name`. Pourquoi ?

.....

.....

▶ 6.6 Contrôle dans le setter

Que fait la condition `if` dans `setName` ? Quel est l'intérêt d'un setter par rapport à un accès direct à l'attribut ?

.....

.....

▶ 6.7 Trace d'exécution

Donnez l'affichage produit par le code suivant :

```
1 Animal a = new Animal("Rex");
2 a.setName("");
3 a.setName("Max");
```

```
4 System.out.println(a.getName());
```

▶ 6.8 À vous de coder

Ajoutez à la classe `Animal` un attribut `age` de type `int`, avec son getter et son setter. Le setter doit refuser les valeurs négatives.

Exercice 7 : Attributs et méthodes `static`

On enrichit la classe `Animal` avec un compteur d'animaux et une méthode utilitaire. La classe `Dog` reste inchangée.

```java
class Animal {
    private String name;
    private static int count = 0;

    Animal(String name) {
        this.name = name;
        count++;
    }

    public String getName() {
        return this.name;
    }

    public void setName(String name) {
        if (isValidName(name)) {
            this.name = name;
        }
    }

    public static int getCount() {
        return count;
    }

    public static boolean isValidName(String name) {
        return name != null && !name.isEmpty();
    }

    void sound() {
        System.out.println("Animal Sound " + this.name);
    }
}
```

▶ 7.1 Attribut de classe

Quel mot-clé fait de `count` un attribut de classe ? Quelle différence y a-t-il avec l'attribut d'instance `name` ?

.....

.....

▶ 7.2 Instance ou `static` ?

Cochez la bonne colonne pour chaque élément de la classe `Animal`.

Élément	Instance	Static
`name`	☐	☐
`count`	☐	☐
`getName()`	☐	☐
`getCount()`	☐	☐
`isValidName()`	☐	☐

▶ 7.3 Copies en mémoire

On crée trois objets `Animal`. Combien de copies de `name` existent-elles ? Combien de copies de `count` ?

.....

.....

▶ 7.4 Appel d'une méthode `static`

Écrivez l'instruction qui, dans `Main`, affiche la valeur du compteur sans passer par un objet.

.....

▶ 7.5 Valeurs

On exécute le code suivant :

```
1 Animal a1 = new Animal("a1");
2 Animal a2 = new Animal("a2");
3 Dog d1 = new Dog("d1");
4 a2 = a1;
```

Que valent ensuite `Animal.getCount()` et `Animal.isValidName("")` ? Expliquez.

.....

.....

▶ 7.6 Restriction

Pourquoi la méthode `getCount()` ne pourrait-elle pas afficher `this.name` ?

.....

.....

► 7.7 La méthode `main`

Pourquoi la méthode `main` est-elle déclarée `static` ?

.....

.....

► 7.8 À vous de coder

Ajoutez à `Animal` une constante de classe `MAX_NAME_LENGTH` valant 20, puis modifiez `isValidName` pour refuser aussi les noms plus longs que cette limite.

CORRECTION : TP 1

Exercice 1 : Analyse des classes

▶ 1.1 Extraction des classes

Correction

Trois classes sont déclarées : `Animal`, `Dog` et `Main`.

▶ 1.2 Attributs

Correction

`Animal`	`String name`
`Dog`	aucun attribut déclaré (`name` est hérité de `Animal`)
`Main`	aucun attribut

▶ 1.3 Méthodes

Correction

`Animal`	`void sound()`
`Dog`	`void sound()` (redéfinie)
`Main`	`public static void main(String[] args)`

▶ 1.4 Constructeurs

Correction

`Animal`	`Animal(String name)`
`Dog`	`Dog(String name)`
`Main`	aucun constructeur explicite (Java fournit un constructeur par défaut)

▶ 1.5 Héritage

Correction

a) La classe `Dog` hérite d'une autre classe.

b) La classe mère (super-classe) est `Animal`.

c) La classe fille (sous-classe) est `Dog`.

d) L'instruction `class Dog extends Animal` (mot-clé `extends`).

▶ 1.6 Redéfinition de méthode

Correction

a) La méthode `sound()`.

b) Dans la classe `Animal`.

c) `@Override` indique au compilateur que la méthode redéfinit une méthode de la classe mère. Le compilateur vérifie alors que la signature correspond bien et signale une erreur en cas de faute (nom ou paramètres différents). Cela améliore aussi la lisibilité du code.

Exercice 2 : Lecture du programme

▶ 2.1 Objets

Correction

4 objets sont créés : deux `Animal` et deux `Dog`.

▶ 2.2 Références

Correction

Les références sont `a1`, `a2` (de type `Animal`) et `d1`, `d2` (de type `Dog`).

▶ 2.3 Constructeurs

Correction

Le constructeur de la classe mère `Animal(String name)` est exécuté en premier, via l'appel `super(name)`. En Java, la partie héritée de l'objet doit être initialisée avant celle de la classe fille : le constructeur de `Dog` ne poursuit son exécution qu'après la fin de celui d'`Animal`. L'affichage est donc `new Animal d1` puis `new Dog d1`.

Exercice 3 : Références et objets

▶ 3.1 et 3.2

Correction

3.1 : ☒ Non (aucun `new`, on copie seulement une référence).

3.2 : ☒ Oui (`a1` et `a2` désignent le même objet).

▶ 3.3 Représentation graphique

Correction

`a1` ⟶ `Animal : name = "a1"`

`a2` ⟶ `(même objet que a1)`

L'ancien objet `Animal("a2")` n'est plus référencé : il devient candidat au *ramasse-miettes* (garbage collector).

Exercice 4 : Analyse de `sound()`

Correction

4.1 : pour `a1` (objet de type `Animal`), c'est la version de la classe `Animal` qui est appelée : `Animal Sound ...`

4.2 : pour `d1` (objet de type `Dog`), c'est la version redéfinie dans la classe `Dog` : `Dog Sound ...`

4.3 : le **polymorphisme** (redéfinition de méthode et liaison dynamique) : la version appelée dépend du type réel de l'objet à l'exécution.

Exercice 5 : Trace d'exécution

Correction

```
new Animal a1
new Animal a2
new Animal d1
new Dog d1
new Animal d2
new Dog d2
Animal Sound a1
Animal Sound new a1 object name
Animal Sound new a1 object name
new Animal d1
new Dog d1
Dog Sound new d1 object name
Dog Sound d1
```

Explications :

— Après `a2 = a1`, les deux références pointent vers le même objet : modifier `a1.name` modifie aussi ce que voit `a2`.

— `d2 = new Dog(d1.name)` crée un *nouvel* objet avec une copie de la valeur `"d1"` (les `String` sont immuables). Changer ensuite `d1.name` n'affecte donc pas `d2`.

Exercice 6 : Encapsulation, getters et setters

▶ 6.1 Modificateur d'accès

Correction

Le mot-clé `private`. L'attribut n'est accessible que depuis l'intérieur de la classe `Animal` : ni `Main`, ni la classe fille `Dog` ne peuvent y accéder directement. C'est le principe d'**encapsulation**.

▶ 6.2 Identification

Correction

Getter	`public String getName()`
Setter	`public void setName(String name)`

▶ 6.3 Rôle

Correction

Le **getter** permet de *lire* la valeur d'un attribut privé. Le **setter** permet de *modifier* cette valeur, éventuellement après vérification. Convention de nommage : `getNomAttribut()` et `setNomAttribut(valeur)`.

▶ 6.4 Accès depuis `Main`

Correction

Non : l'attribut est `private`, donc l'instruction provoque une erreur de compilation (`name has private access in Animal`). On utilise le setter :

```
a1.setName("new a1 object name");
```

▶ 6.5 Classe `Dog`

Correction

Un attribut `private` n'est pas accessible dans les classes filles, même s'il fait partie de l'objet. `Dog` doit donc passer par la méthode publique héritée `getName()`.

▶ 6.6 Contrôle dans le setter

Correction

La condition refuse les valeurs `null` ou vides : dans ce cas, l'ancien nom est conservé. Le setter permet donc de **valider les données** et de garder l'objet dans un état cohérent. Il permet aussi de modifier l'implémentation interne de la classe sans changer le code qui l'utilise.

▶ 6.7 Trace d'exécution

Correction

```
new Animal Rex
Max
```

L'appel `a.setName("")` est ignoré (chaîne vide) ; `a.setName("Max")` est accepté.

▶ 6.8 À vous de coder

Correction

```java
private int age;

public int getAge() {
    return this.age;
}

public void setAge(int age) {
    if (age >= 0) {
        this.age = age;
    }
}
```

Exercice 7 : Attributs et méthodes `static`

▶ 7.1 Attribut de classe

Correction

Le mot-clé `static`. Un attribut `static` appartient à la **classe** : il est partagé par tous les objets. Un attribut d'instance (`name`) appartient à chaque **objet** et possède une valeur propre à chacun.

▶ 7.2 Instance ou `static` ?

Correction

`name`	☒	☐
`count`	☐	☒
`getName()`	☒	☐
`getCount()`	☐	☒
`isValidName()`	☐	☒

▶ 7.3 Copies en mémoire

Correction

Trois copies de `name` (une par objet) mais **une seule** copie de `count`, stockée au niveau de la classe et partagée par tous les objets.

▶ 7.4 Appel d'une méthode `static`

Correction

On appelle la méthode avec le nom de la classe :

```
System.out.println(Animal.getCount());
```

▶ 7.5 Valeurs

Correction

`Animal.getCount()` vaut **3** et `Animal.isValidName("")` vaut **false**. Trois objets sont créés (deux `Animal` et un `Dog`) ; la construction d'un `Dog` appelle `super(name)`, donc le constructeur d'`Animal` incrémente aussi `count`. L'instruction `a2 = a1` ne crée ni ne supprime de compteur : elle ne change pas `count`. Enfin, la chaîne vide est refusée par `isValidName`.

▶ 7.6 Restriction

Correction

Une méthode `static` appartient à la classe et s'exécute sans objet courant : il n'y a donc pas de `this`. Elle ne peut accéder directement qu'aux membres `static`. Pour utiliser `name`, il faudrait lui fournir une référence vers un objet précis.

▶ 7.7 La méthode `main`

Correction

La machine virtuelle Java doit pouvoir lancer le programme *avant* qu'un objet existe : `main` est donc `static` et s'appelle directement sur la classe `Main`.

▶ 7.8 À vous de coder

Correction

```java
public static final int MAX_NAME_LENGTH = 20;

public static boolean isValidName(String name) {
    return name != null && !name.isEmpty()
           && name.length() <= MAX_NAME_LENGTH;
}
```

Le mot-clé `final` rend la valeur non modifiable : `static final` définit une constante de classe (par convention en majuscules).